



El V4 del Grupo 5 — la propuesta del sándwich y qué nos cambia

Estado: ABIERTO. Nada de lo que dice este documento está decidido.

Qué es. El documento único de la integración con el Grupo 5 (Tema 05, Desafíos Prácticos). Contiene la propuesta completa del **sándwich** —el contrato del contenedor partido en dos capas— y el análisis del `Propuesta_Integracion_G5_G6_Entrypoint_V4.pdf` (6 páginas, septiembre 2026) contra ella. Es autocontenido: no hace falta leer nada más para entenderlo.

Para quién. Los once.

Qué sale de acá. La respuesta que se les manda vive en `otros/Respuesta_G8_a_Propuesta_V4.md`.

Última revisión: **7 de septiembre de 2026.**

Índice

1. De dónde viene el V4
2. La propuesta: el sándwich
3. El diff fino: su capa 1 contra nuestra tapa
4. Las cuatro ausencias que son agujeros
5. Las dos contradicciones directas
6. El problema serio: qué pasa con D6
7. Qué se cae de nuestros documentos
8. El catálogo de perfiles
9. Las opciones 1 y 3 son la misma opción
10. Cómo se prueba que no rompimos nada
11. Trabajo que aparece y trabajo que se borra
12. Decisiones nuevas para el README
13. Qué hacemos ahora

1. De dónde viene el V4

El pedido de fondo del Grupo 5 es que **nuestro contenedor no sepa nada del ejercicio**. Hoy `sandbox/runner/entrypoint.sh` sabe Java: llama a `javac` en dos fases, arma el classpath, deriva los nombres de clase e invoca el `ConsoleLauncher` de JUnit. Con eso, cada tipo de desafío nuevo —ArchUnit, PMD, completado de código— es un cambio en **nuestro** código y un redespigue nuestro. El acoplamiento es real y la queja es legítima.

El V2 proponía dos caminos: **A**, inyectar un `run.sh` en el tar y poner `ENTRYPOINT ["/bin/sh", "run.sh"]`; **B**, un

array `commands` en el contrato. La opción A no se puede implementar tal cual, por dos motivos estructurales:

1. **El `run.sh` todavía no existe cuando el contenedor arranca.** Viene adentro del tar, y el tar entra por `stdin` después de que el contenedor ya está corriendo. Un `ENTRYPOINT` que apunta a un archivo que no está falla al instante.
2. **Se perdería el nonce.** El nonce es la única defensa contra la falsificación del reporte, y entregarle el `ENTRYPOINT` al Grupo 5 significa que el secreto tiene que pasar por el script que convive con el código del alumno. Un secreto que atraviesa la zona hostil deja de ser secreto.

El V4 cambia tres cosas de fondo:

	V2	V4
Entrypoint	Piden que sea el <code>run.sh</code> de ellos	Retiran el pedido. El entrypoint "siempre pertenecerá al Grupo 8", en dos capas
Opciones	2 (script universal / array de comandos)	3 , y la que ponen primera es nueva: catálogo de perfiles guardado en nuestra base
Imágenes	No se trataba	§6: nosotros construimos las imágenes base con <code>/libs</code> preinstalado y usuario no-root

El §3 del V4 describe dos capas: capa 1 de G8 (lee el nonce por `stdin`, extrae el tar, imprime la marca de inicio, llama a la capa de evaluación, imprime la marca de fin) y capa 2 de G5 (compila, testea, devuelve un JSON, y **desconoce por completo la existencia del nonce**).

Es la misma forma que nuestra propuesta. Lo que sigue es nuestra propuesta completa; después, el diff.

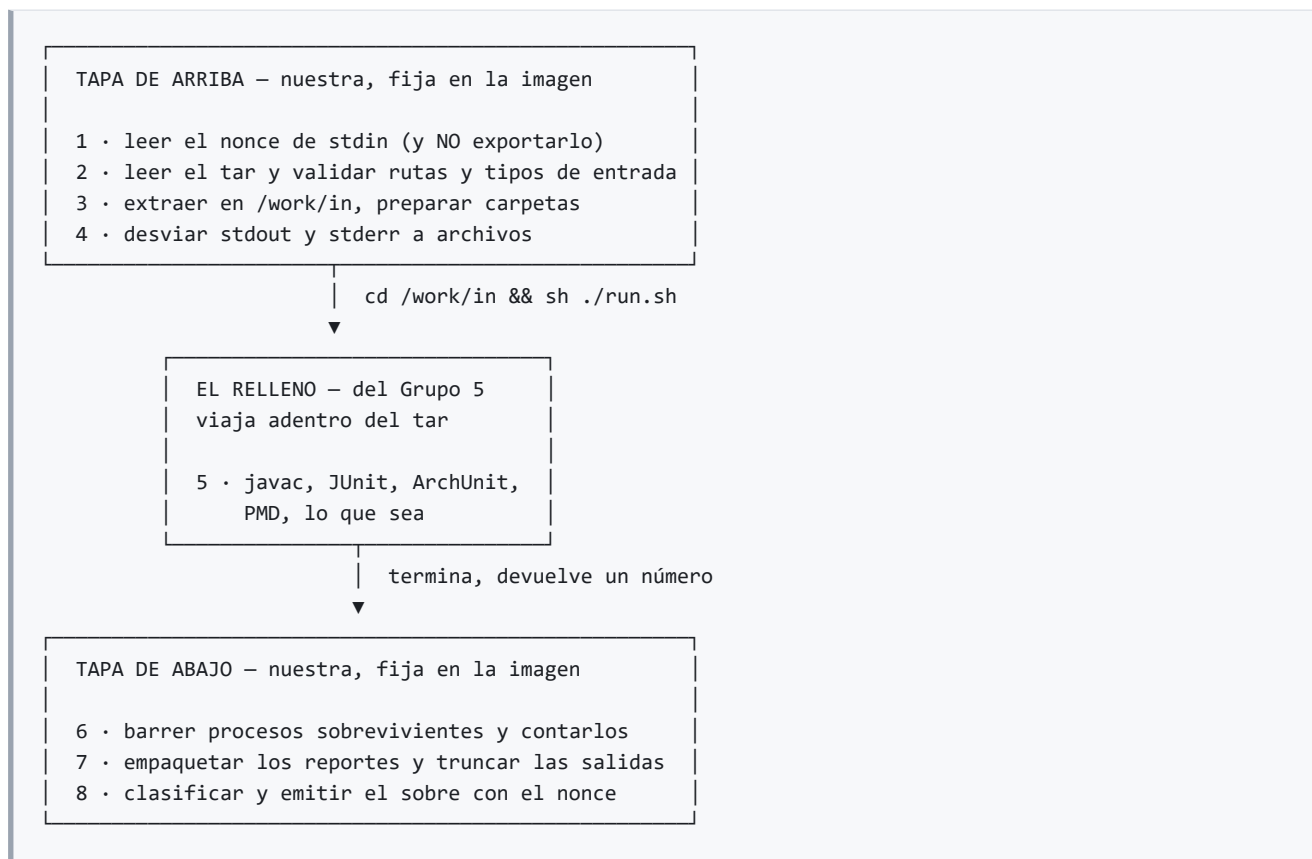
2. La propuesta: el sándwich

2.1 La idea en una frase

El entrypoint deja de ser **el pipeline de Java** y pasa a ser **la cáscara que hace confiable a cualquier pipeline**. El Grupo 5 escribe el relleno; nosotros conservamos las dos tapas.

Les damos lo que piden —que no sepamos nada del ejercicio— sin perder ninguna de las propiedades de seguridad, porque **esas propiedades no viven en el medio, viven en los extremos**.

2.2 El dibujo



Todo lo que hoy está cableado en el medio de `sandbox/runner/entrypoint.sh` — `javac`, la derivación de nombres de clase, el `ConsoleLauncher` — **sale de nuestra imagen** y se va a vivir al `run.sh`.

2.3 Qué gana cada uno

	Antes	Con el sándwich
G5	Depende de nosotros para cada tipo de desafío nuevo	Escribe el <code>run.sh</code> y listo
Nosotros	Nuestro script sabe de JUnit, de classpaths, de compilación	Nuestro script no sabe qué lenguaje es
Seguridad	Todas las guardas en nuestro script	Idéntica: las guardas están en las tapas

2.4 El contrato, en tres preguntas

Adentro del contenedor hay **dos programas escritos por dos grupos que nunca se hablan entre sí**. Corren uno después del otro. No hay HTTP, no hay llamadas a funciones, no hay parámetros: lo único que comparten es **el disco del contenedor y un número al final**.

Todo el contrato cabe en las tres preguntas que el Grupo 5 se va a hacer cuando se siente a escribir su script: qué me encuentro, dónde dejo lo que produzca, cómo aviso cómo me fue.

Pregunta 1 — qué se encuentra `run.sh` cuando arranca

```

/work/          ← lo ÚNICO escribible. Vive en RAM, muere con el contenedor
├── in/          ← acá extrajimos el tar. run.sh arranca parado en esta carpeta
│   ├── run.sh   su propio script
│   ├── src/Solution.java el código del alumno
│   └── test/SolutionTest.java las pruebas del profesor
├── reports/     ← vacía. Es el buzón de salida
├── tmp/         ← vacía. Borrador, no vuelve
└── status/      ← vacía. Opcional

/libs/          ← junit.jar, pmd, archunit... SOLO LECTURA, viene en la imagen
/               ← todo el resto del disco: SOLO LECTURA

```

Las rutas se pasan también como variables de entorno. **Son apodos de las carpetas de arriba, no son el contrato en sí:**

Apodo	Carpeta	Para qué
<code>SANDBOX_IN</code>	<code>/work/in</code>	dónde está el código; es el directorio de trabajo
<code>SANDBOX_REPORTS</code>	<code>/work/reports</code>	dónde dejar lo que tiene que volver
<code>SANDBOX_TMP</code>	<code>/work/tmp</code>	borradores
<code>SANDBOX_STATUS</code>	<code>/work/status</code>	avisos opcionales
<code>SANDBOX_LIBS</code>	<code>/libs</code>	dónde están las herramientas
<code>SANDBOX_MEM_MB</code>	<code>512</code>	cuánta memoria hay, para dimensionar la JVM

¿Y estas no las lee el alumno, igual que leería el nonce? Sí, las lee. La diferencia es que **no son secretas**: saber que los reportes van a `/work/reports` no le sirve de nada al que quiere hacer trampa. Esa asimetría es exactamente el motivo por el que el nonce viaja por otro lado y estas no.

Pregunta 2 — dónde deja lo que produce

Una sola regla, y es la más corta del documento:

Todo lo que quede en `/work/reports` vuelve. Todo lo demás se pierde.

Empaquetamos esa carpeta entera y la metemos adentro del sobre. **No la miramos, no la interpretamos, no sabemos qué hay adentro**: la mandamos. Si es un XML de JUnit, bien; si es uno de PMD, también; si son tres archivos, van los tres.

Y el corolario: **si esa carpeta queda vacía, no hay aprobación posible**. Sin evidencia no hay veredicto.

Pregunta 3 — cómo avisa cómo le fue

Con el número con el que termina. Y con la aclaración más importante de esta sección:

El número no es el veredicto. El veredicto sale del reporte, siempre. El número solo dice **qué tan lejos llegó el script**.

Rango	Dueño	Qué significa
0	G5	"llegué hasta el final, mirá el reporte"
40-59	G5	"me frené a propósito, y este número dice por qué" — la tabla la definen ellos
20-31	nosotros	reservados, los doce que ya usa <code>codigo_de()</code> hoy
32-39	—	hueco a propósito, para crecer sin renegociar
cualquier otro	—	se traduce a <code>ERROR_INTERNO</code>

Por qué las bandas. Un código de salida es **un byte**, y buena parte ya tiene dueño por convención de Unix: el 0 es universal; el 1 y el 2 los devuelven `javac`, `java`, `PMD` y el propio `sh` ante cualquier error; el 126 es "lo encontré pero no lo puedo ejecutar" —que es exactamente el error del `noexec`, ver abajo—; el 127 es "no encontré el comando"; y 128+N es muerte por señal (137 sin memoria, 143 SIGTERM, 152 se acabó la CPU). El espacio realmente libre va del 3 al 125. Si el Grupo 5 usara 1 para "no compila", no lo podríamos distinguir de "el shell se rompió".

Opcionalmente, si quieren que el alumno vea "no compila" en vez de "error interno", pueden dejar `$SANDBOX_STATUS/fase` con una etiqueta (`COMPILACION`, `PRUEBAS`, `ANALISIS` ...) y `$SANDBOX_STATUS/detalle` con una línea de diagnóstico. Los copiamos al sobre tal cual.

¿Para qué el número, si ya está el archivo? Porque el archivo no siempre llega, y el número sí. Si el `run.sh` se muere de golpe —lo mató el kernel por consumir CPU, tuvo un error de sintaxis— no alcanzó a escribir nada. El número igual nos llega, porque lo produce el sistema operativo, no el script.

El archivo es el canal **rico pero frágil**. El número es el canal **pobre pero indestructible**. Por eso están los dos, y por eso el obligatorio es el número.

Y los códigos de la banda 40-59 no nos los piden: nos los informan. No implementamos nada por cada código ni sabemos qué significan. El número atraviesa nuestro lado sin que nadie lo mire:

```
run.sh termina con 42
→ el entrypoint lo copia al sobre como exitRunSh: 42
→ el worker lo copia al resultado de la ejecución
→ vuelve al Grupo 5, que es quien sabe que 42 significa "PMD se rompió"
```

El que puso el número es el mismo que lo lee; nosotros somos el cartero. De ahí sale el mejor test de si el diseño está bien: **si el Grupo 5 agrega un código nuevo y nosotros tenemos que cambiar algo —una línea, una tabla, un `if`—, el sándwich falló.**

2.5 La película completa de una ejecución

Momento	Quién manda	Qué pasa
t0	el ejecutor, afuera del contenedor	crea el contenedor con la spec fija (sin red, disco de solo lectura, 512 MB) y lo arranca
t1	nuestro entrypoint	lee la primera línea de stdin: el nonce. Lo guarda en una variable que no exporta
t2	nuestro entrypoint	lee el resto de stdin: el tar. Valida rutas y tipos de entrada <i>antes</i> de abrirlo

Momento	Quién manda	Qué pasa
t3	nuestro entrypoint	extrae en <code>/work/in</code> ; crea <code>reports/</code> , <code>tmp/</code> , <code>status/</code>
t4	nuestro entrypoint	desvía stdout y stderr a archivos y fija el presupuesto de CPU
t5	<code>run.sh</code> , del Grupo 5	compila, ejecuta, analiza —lo que sea— y deja lo suyo en <code>/work/reports</code>
t6	nuestro entrypoint	barre los procesos que hayan quedado vivos y los cuenta
t7	nuestro entrypoint	empaqueta <code>/work/reports</code> , trunca las salidas si se pasaron de largo
t8	nuestro entrypoint	emite el sobre rodeado por el nonce y termina; el contenedor muere

Fijate en t4, que es la parte menos obvia. Antes de llamar al `run.sh` desviamos su salida a archivos. Por eso `run.sh` físicamente no puede escribir en la salida real del contenedor: todo lo que imprima, y todo lo que imprima el alumno, cae en un archivo que nosotros después leemos y recortamos. **El único que escribe en la salida real es el entrypoint, en t8.** Eso es lo que hace que el sobre no se pueda contaminar, y es la otra mitad de la defensa del nonce.

Y fijate que **el Grupo 5 ocupa una sola fila de esa tabla.** Todo el resto es cáscara nuestra, idéntica para cualquier desafío.

2.6 Qué se lleva el entrypoint cuando `run.sh` termina

Qué quedó en el disco	Dónde aparece en el sobre
<code>/work/reports/</code> entera	<code>reportesTarGzB64</code> — comprimida, sin mirar el contenido
lo que <code>run.sh</code> y el alumno imprimieron	<code>stdoutB64</code> y <code>stderrB64</code> , ya truncados
el número con que terminó <code>run.sh</code>	<code>exitRunSh</code> , y traducido a <code>resultado</code>
<code>/work/status/fase</code> , si existe	<code>faseDeclarada</code>
procesos que quedaron vivos en t6	<code>procesosSobrevivientes</code>
<code>/work/tmp/</code>	nada: se pierde con el contenedor

2.7 Las cinco cosas que `run.sh` no puede hacer

No puede...	Por qué
Escribir el sobre	No tiene el nonce y su salida está desviada a un archivo (t4)
Dejar procesos corriendo sin esperarlos	El barrido de t6 los cuenta y arruina el veredicto
Escribir fuera de <code>/work</code>	Todo el resto del disco es de solo lectura
Usar la red	El contenedor corre sin red. No hay <code>curl</code> , no hay Maven, no hay descargas
Elegir imagen, memoria o tiempos	Invariante P1 . Eligen perfil , de una lista (§2.9)

Las últimas dos merecen énfasis en la respuesta: **todo lo que su script necesite tiene que estar ya adentro de la imagen.** No se puede bajar en el momento.

Y una trampa que va en la primera línea de la respuesta. `/work` está montada con `noexec` : el sistema opera-

tivo se niega a ejecutar archivos que estén ahí. Es la defensa contra el alumno que sube un binario. Consecuencia:

```
sh ./run.sh      # OK: el que ejecuta es sh, y él solo LEE el archivo
./run.sh        # falla siempre: Permission denied
```

Es el típico error que aparece el día de la demo.

Si su `run.sh` lanza algo en segundo plano con `&` y no lo espera, nuestro barrido lo va a contar como proceso sobreviviente y **todas las entregas van a volver marcadas como veredicto no confiable**. Tienen que esperar a todo lo que lancen.

2.8 Lo que no se mueve, y por qué

Estas guardas se ganaron corriendo la suite de casos hostiles. **No pueden mudarse al `run.sh`**, por un motivo que se entiende de una: protegen contra el alumno, y el alumno corre *adentro* del `run.sh`. Ponerlas ahí sería pedirle al vigilado que se vigile.

Guarda	Qué ataque frena	Por qué queda de nuestro lado
Barrido de procesos sobrevivientes	El alumno deja un programa en segundo plano que reescribe el reporte después de que las pruebas terminaron	Tiene que correr después de que <code>run.sh</code> devuelve el control
Truncado de la salida adentro del contenedor	El alumno imprime gigabytes para tumbar al worker	Si truncáramos afuera, ya nos habríamos comido la memoria
Nonce y marcadores	Falsificación del reporte por stdout	<code>run.sh</code> ni lo ve
Detección de falta de memoria	Distinguir "se quedó sin RAM" de "el código falló"	El dato lo da Docker al ejecutor, no el contenedor

Y una cosa que sí perdemos, y hay que decirla. Hoy contamos cuántas pruebas declara el reporte de JUnit y usamos ese número como red: *"si el reporte dice 0 pruebas, no puede haber aprobado"*. Ese conteo lee el XML de JUnit, y **para un reporte de PMD no significa nada**. Fuera de los perfiles de JUnit, ese número deja de existir y la red se muda al worker. Es el punto que la §6 convierte en decisión.

2.9 Perfiles en vez de imagen libre

El contrato del V4 deja que el cliente elija la imagen y los tiempos, y eso rompe la invariante **P1**: nadie de afuera fija los recursos del sandbox. El arreglo es simple: en vez de mandarnos el nombre de una imagen, nos mandan **el nombre de un perfil**, elegido de una lista cerrada que vive de nuestro lado.

```
perfil "java21-tools" → imagen sandbox-java:21-tools, 512 MB, 1 CPU, 128 procesos, tmpfs de 64 MB
perfil "java21-basico" → imagen sandbox-runner:1.0.0, 512 MB, 1 CPU, 128 procesos, tmpfs de 64 MB
```

Ellos **eligen** de la lista; no **escriben** la lista.

Las dos clases de pedido

Acá está el corazón de la propuesta. **Hay dos tipos de cosas que el Grupo 5 puede querer, y se comportan de manera completamente distinta.**

	Clase 1: no nos piden nada	Clase 2: necesitan un release nuestro
Qué	cambiar los comandos, agregar un tipo de desafío, cambiar el formato del reporte, definir un código de salida nuevo, cambiar cómo compilan, agregar una fase	agregar una herramienta a <code>/libs</code> , crear un perfil nuevo, subir la memoria o el tiempo, cambiar la imagen base
Cómo se hace	lo escriben en el <code>run.sh</code> y lo mandan en el paquete	nos lo piden, reconstruimos la imagen y desplegamos
Cuánto tardan	lo que tarden en escribirlo	lo que tardemos nosotros
Cuántas veces por cuatrimestre	muchas, todo el tiempo	pocas

La apuesta de todo el sándwich es que **la Clase 1 es la que ocurre seguido**. Un ejercicio nuevo, un cambio en los tests, otra forma de armar el classpath: eso pasa todas las semanas, y hoy cada una de esas cosas es un release nuestro. Ahí está el acoplamiento que denuncian, y ahí está el 90% del dolor.

Con esto hay que corregirles una frase del PDF. Dicen "mantenimiento cero" para nosotros. Es cierto para la Clase 1. Pero si la cátedra pide una herramienta que no está instalada en la imagen, hay que reconstruir la imagen, y la imagen es nuestra. Conviene decirlo nosotros antes de que aparezca solo en la defensa. Y decirles lo otro: **estamos en planeamiento, no hay nada desplegado ni alumnos usando el sistema, así que agregar herramientas ahora no nos cuesta casi nada. Que pidan todo lo que crean que van a necesitar.**

2.10 Los relojes: acá está el verdadero costo

Hay **cinco relojes**, en capas de adentro hacia afuera. Los valores son los que están hoy en la imagen y en la spec:

#	Reloj	Valor	Dónde vive	Qué tapa
1	Compilación, pared	20 s	entrypoint	un <code>javac</code> que se vuelve loco con código patológico
2	Pruebas, CPU	10 s	entrypoint (<code>ulimit -t</code>)	el presupuesto del alumno: el bucle infinito
3	Pruebas, pared	30 s	entrypoint	el que <i>duerme</i> en vez de quemar CPU
4	Contenedor, CPU	20 s	spec del contenedor	red del kernel por si falla el entrypoint
5	Ejecución entera, pared	60 s	el ejecutor, afuera	red final: por si falla todo lo de adentro

Del 1 al 3 los aplica **nuestro script**, porque sabe en qué fase está parado. El 4 y el 5 son redes de afuera hacia adentro y no cambian con esta propuesta.

El reloj 2 se mide en CPU, no en pared, porque con reloj de pared dos entregas idénticas dan resultados distintos según la carga del host: medimos entre 1,8 y 5,2 segundos para el mismo bundle. En CPU es estable.

Y compilar no se le cobra al alumno, por una razón de justicia: compilar no es el trabajo que estamos evaluando, y cuánto tarda `javac` depende de cuántos archivos armó el profesor. Si saliera del presupuesto del alumno, un ejercicio con 30 archivos de prueba le dejaría menos segundos para su algoritmo que uno con 3. Dos alumnos igual de buenos, distinto presupuesto, por una decisión del profesor.

Qué se rompe con el sándwich. Hoy el entrypoint ejecuta las fases él mismo, así que sabe dónde está parado. Con el sándwich ejecuta *una sola cosa opaca* y no puede saber cuándo termina de compilar ni cuándo empieza a probar.

Solo puede poner un techo alrededor de todo — y ahí compilar vuelve a salir del bolsillo del alumno.

	Cómo funciona	Qué cuesta
(a) Un solo techo	un presupuesto de CPU para todo el <code>run.sh</code>	se pierde "compilar es gratis"
(b) El Grupo 5 subdivide	ellos ponen sus relojes por fase y avisan por <code>\$_SANDBOX_STATUS/fase</code>	se recupera todo, pero depende de que se tomen el trabajo
(c) Techo holgado	un solo techo, pero generoso: si compilar gasta ~2 s y el presupuesto es 20 s, la distorsión es del 10%	casi nada, y es simple

Propuesta: (c) como piso, (b) como opción. Lo que **no** hay que hacer es prometer (b) como obligatorio: sería trasladarles un requisito nuestro disfrazado de contrato.

Lo más sutil de todo: `ulimit -t 20` **no limita "el contenedor", limita un proceso.** El límite se hereda a los hijos, pero **el contador no**: cada programa nuevo arranca en cero. Hay que medirlo antes de escribirle un número a nadie (A1, §12).

3. El diff fino: su capa 1 contra nuestra tapa

Nuestro `sandbox/runner/entrypoint.sh` (456 líneas) ya está estructurado como el sándwich, aunque todavía tenga el relleno de Java cableado adentro:

```

paso 1  leer nonce + leer y VALIDAR el tar + extraer           ← tapa de arriba
paso 2  javac de la solución
paso 3  javac de la suite                                     }
paso 4  derivar nombres de clase sin escanear                 } el relleno: sale de nuestra
paso 5  correr JUnit con ulimit -t                           } imagen y se va al run.sh de G5
paso 6  barrer y contar procesos sobrevivientes              }
paso 7  empaquetar reportes, truncar salidas,                 } tapa de abajo
        clasificar y emitir el sobre por fd 3                }
```

Contra eso, la capa 1 del V4 §3 tiene cinco pasos. El mapeo:

Nuestro	Su capa 1 (§3)	Estado
leer nonce en variable no exportada (R7.3)	"leer el nonce secreto provisto por el Ejecutor a través de stdin"	≈ coincide, sin el mecanismo
leer el tar y validar rutas y tipos de entrada	"extraer el TAR de archivos en memoria"	✗ falta la validación
extraer en <code>/work/in</code> , preparar <code>/work/{reports,tmp,status}</code>	(no fija rutas)	! sin layout acordado
desviar stdout/stderr del hijo a archivos	—	✗ no existe
<code>cd /work/in && sh ./run.sh</code>	"llamar a la capa de evaluación"	≈ coincide
marca de inicio / de fin	idénticas a las nuestras	✓ coincide
barrer y contar procesos sobrevivientes	—	✗ no existe

Nuestro	Su capa 1 (§3)	Estado
empaquetar <code>/work/reports</code> , truncar, clasificar, emitir el sobre	—	X no existe

De los siete pasos, su capa 1 tiene tres, esboza dos y le faltan cuatro. **Los cuatro que faltan son los que sostienen las garantías.**

4. Las cuatro ausencias que son agujeros

Cada una se lee en dos partes: **qué falta** en su capa 1, y **cómo lo cubrimos** de nuestro lado. La segunda parte es la que importa, porque casi todas se resuelven sin que ellos toquen nada — son cosas que viven en la tapa por diseño. El resumen está en §4.5.

4.1 Extraen el tar sin validarlo

El V4 dice "extraer el TAR en memoria" y sigue. Nuestro paso 1 valida **rutas** (relativas, sin `..`, sin barra inicial) y **tipos de entrada** — symlinks y hard links, que son la clase de bug que tumbó a Judge0 tres veces (hallazgo [IE] #4, pendiente **P2**).

Y encima el payload del V4 manda `"path": "src/main/java/com/tup/Solution.java"` desde el cliente: **el traversal lo mandan ellos y lo tenemos que rechazar nosotros**, en dos lugares — la API con `RUTA_INVALIDA` (que todavía no existe: es el Abierto 2 del `07`) y el entrypoint como defensa en profundidad. En su modelo no está en ninguno de los dos.

Cómo lo cubrimos. La validación es del paso 1 de la tapa y se queda ahí: no depende de que ellos escriban nada. Lo que sí cambia de nuestro lado es **cómo** valida. Hoy el validador acepta sólo rutas que empiecen con `src/` o `test/`, y esa lista blanca no sobrevive al sándwich —con `run.sh` en la raíz y configuraciones del Grupo 5 en cualquier lado, el árbol se abre—. Hay que reemplazarla por **prohibiciones sobre el tipo de entrada del tar**: nada de symlinks, nada de hard links, nada de rutas absolutas ni con `..`. Es el pendiente **P2**, y pasa de deseable a bloqueante: mientras la lista blanca era angosta el hueco de los enlaces era estrecho; abriendo el árbol, deja de serlo.

En la respuesta se los mencionamos como recomendación —que su API tampoco acepte rutas raras— pero **no dependemos de que lo hagan**. Es defensa en profundidad, no delegación.

4.2 El modelo de confianza de las marcas es el opuesto al nuestro

Esta es la diferencia más importante, y es fácil que pase desapercibida porque **las marcas se ven iguales**.

- **Ellos:** `stdout` queda vivo y las marcas **delimitan una región** de ese `stdout`. Lo que la capa 2 imprima entre las marcas es el contenido que vuelve.
- **Nosotros:** `t4` desvía `stdout` y `stderr` del hijo a archivos. La capa 2 **nunca toca el `stdout` real**. El sobre lo construye la tapa de abajo y sale por **fd 3**, con las salidas del alumno adentro, en base64 y truncadas.

El nonce impide falsificar las marcas en los dos modelos. Pero en el de ellos no separa la carga del ruido: cualquier cosa que el código del alumno imprima cae adentro del sobre. En el nuestro no puede entrar nada que no hayamos puesto nosotros. **Mismo mecanismo, garantía distinta.**

Cómo lo cubrimos. El desvío es de la tapa (`t4`) y ya está implementado: no hay nada que negociar para que ocurra. Su script *físicamente no puede* escribir en la salida real del contenedor, escriban lo que escriban.

Pero **acá está la única cosa del V4 que sí les cambia el diseño**, y hay que decirla con todas las letras: si su capa 2 devuelve el JSON de la nota por `stdout`, ese JSON **no llega como resultado** — cae en `stdoutB64` junto con lo que haya impreso el alumno, mezclado y truncado. El JSON tiene que ir como un archivo más a `/work/reports`.

Es el punto 4 de la lista de pedidos de la respuesta, y el único que no podemos resolver solos.

4.3 No hay tapa de abajo

Su paso 5 es imprimir la marca de fin, y nada más. Se caen las tres guardas de §2.8 que hoy están implementadas y probadas: el barrido de procesos sobrevivientes, el truncado de salidas y el empaquetado del buzón.

Vale notar que el empaquetado **ya es genérico**: el entrypoint hace `tar -czf - -C /work/reports .` y no mira adentro. Esa parte del sándwich ya está construida y funciona sin saber qué lenguaje corrió.

Cómo lo cubrimos. Es la ausencia más fácil de todas: **los tres pasos ya existen y se quedan donde están**. El refactor de **P9** saca los pasos 2–5 del entrypoint y no toca los 6–8; lo único que cambia es que el sobre deja de tener campos con nombre de Java (`exitCodeJava`, `clasesTest`, `testsEnReporte`) y pasa a tener los genéricos de §2.6 (`exitRunSh`, `faseDeclarada`).

No hay nada que pedirles ni nada que acordar: es cáscara nuestra, invisible desde su lado. En la respuesta va como descripción —para que sepan qué les recogemos y qué no— no como negociación.

4.4 No existe el contrato de códigos de salida

El V4 no menciona los exit codes en ninguna parte. Todas las bandas de §2.4 no tienen contraparte. Es el canal que sobrevive cuando el script se muere de golpe.

Cómo lo cubrimos. La traducción de código a `resultado` la hace la tapa y ya está implementada para nuestra banda 20–31. Lo que agrega el sándwich es una regla de una línea: **todo lo que venga en 40–59 se copia al sobre como `exitRunSh` y se marca como "detenido por la capa de evaluación", sin interpretarlo**.

Eso significa que **el mecanismo funciona aunque el Grupo 5 nunca nos mande su tabla**: sin tabla, el alumno ve un mensaje genérico en vez de *"no compila"*; con tabla, ve el bueno. Degrada bien. Por eso la tabla es **A3** —un insumo que mejora la experiencia— y no un bloqueante del contrato.

Lo que sí es nuestro y hay que escribir: que el hueco 32–39 quede reservado, y que cualquier código fuera de las bandas conocidas caiga en `ERROR_INTERNO` en vez de confundirse con un veredicto.

4.5 Resumen: quién resuelve cada ausencia

Ausencia	La cubre	Requiere algo de ellos
4.1 Validación del tar	Nuestra tapa (paso 1), con P2 pagado antes	No — sólo una recomendación
4.2 Desvío de <code>stdout</code>	Nuestra tapa (t4), ya implementado	Sí: el JSON va al buzón, no a <code>stdout</code>
4.3 Tapa de abajo	Nuestra tapa (pasos 6–8), ya implementados	No
4.4 Códigos de salida	Nuestra tapa, con la regla de la banda 40–59	No para funcionar; sí para que el mensaje sea bueno (A3)

De las cuatro, tres las resolvemos solos y una necesita una línea de acuerdo. Ese es el tamaño real del riesgo de esta integración, y conviene tenerlo presente antes de la reunión: no estamos negociando cuatro cosas, estamos negociando una y avisando de tres.

5. Las dos contradicciones directas

5.1 Su Dockerfile le da `/libs` escribible al usuario del sandbox

```
RUN chown -R sandboxuser:sandboxgroup /app /libs /sandbox-entrypoint.sh
USER sandboxuser
```

El código del alumno corre como `sandboxuser`. Con eso puede reescribir `junit.jar`, `archunit.jar` o el PMD **entre fases** — compilar bien, correr los tests, y reemplazar el jar del analizador antes de que corra. Nuestro §2.4 dice `/libs` **sólo lectura**, y todo el disco salvo `/work` sólo lectura.

El `chown` del propio entrypoint no le suma nada y también sobra. Y falta `/work` entero: tienen `WORKDIR /app`, que no es tmpfs y no existe en nuestro layout.

No rompe el aislamiento —el contenedor sigue sin red, sin root y con límites— pero sí rompe la integridad de las herramientas con las que se dicta el veredicto.

Hay un tercer detalle en el mismo Dockerfile: crean el usuario con `adduser -S sandboxuser`, que **no da el usuario número 1000**. Nuestro contenedor corre como el 1000 y la única carpeta escribible está creada a su nombre. Si no coinciden, **el contenedor muere con `Permission denied` antes de leer el paquete**. Ya nos pasó una vez y está documentado en 08 §4.2.

Cómo lo cubrimos. Las imágenes **las construimos nosotros** (§6 del propio V4), así que las tres cosas se arreglan solas al escribir nuestro Dockerfile: `/libs` y el entrypoint quedan de sólo lectura para el usuario del sandbox, `/work` se monta como tmpfs con `noexec`, y el usuario se crea con uid 1000 explícito. **No hace falta que ellos cambien nada**; el Dockerfile del V4 es un ejemplo, no el artefacto que se despliega.

Lo mencionamos en la respuesta igual, por dos motivos: que el Dockerfile del documento no quede como referencia para nadie, y que sepan por qué la imagen que reciben no va a ser idéntica a la que escribieron. Va en tono de nota técnica, no de corrección.

5.2 `limits.timeoutMs` reintroduce el reloj de pared del cliente

Choca con las dos cosas de §2.10: el reloj del alumno tiene que ser de **CPU**, y **hay cinco relojes, no uno**. Además, como su capa 1 no tiene frontera de fases, el "compilar es gratis" queda inalcanzable salvo por las salidas (b) o (c).

Cómo lo cubrimos. Los relojes 4 y 5 son del ejecutor y de la spec del contenedor: existen pase lo que pase, y ninguno depende del payload. Sobre el `timeoutMs` que mandan, la regla es la de siempre: **el sandbox recorta contra el techo de la plataforma**. Un valor del cliente nunca puede agrandar el presupuesto, sólo achicarlo.

Con la Opción 1 el problema desaparece por construcción, porque el techo pasa a vivir en el perfil (**D18**). Mientras tanto el campo queda aceptado y degradado a sugerencia — no hace falta que lo saquen del contrato para que esto sea seguro.

Lo único que **no** se resuelve solo es el "compilar es gratis": ahí sí necesitamos elegir entre (b) y (c) de §2.10, y (c) —techo holgado— no requiere nada de ellos. Es la razón por la que es la propuesta de piso.

6. El problema serio: qué pasa con D6

Es el punto de mayor impacto de todos, y §2.8 ya lo anticipaba como pérdida declarada. Acá se convierte en decisión.

D6 dice: el veredicto sale del XML de JUnit con `tests > 0`, nunca del exit code. Está cerrado desde el 27-ago y es la defensa que descubrimos cuando verificamos que `System.exit(0)` aprobaba. Hoy la tapa de abajo cuenta los tests del reporte y emite `testsEnReporte` en el sobre.

Bajo el sándwich la tapa no puede hacer eso, porque no sabe qué es un XML de JUnit — ni tiene por qué saberlo, si mañana el reporte es de PMD o de ArchUnit. Si lo dejamos así, el veredicto vuelve a depender de lo que diga la capa 2, que es código que convive con el del alumno. **D6 se rompe.**

La salida es mudarlo, no perderlo:

La verificación de "hay evidencia real" se muda del endpoint al worker. El worker ya recibe `reportesTarGzB64` y ya normaliza los reportes (§4 §6). El **perfil** declara qué formato de reporte produce (`junit-xml`, `pmd-xml`, ...), y el worker aplica la regla de D6 contra ese formato.

Por qué esto conserva la propiedad y no es un parche: el worker corre **afuera del contenedor**, fuera del alcance del código del alumno. La regla sigue siendo "sin evidencia no hay aprobación", sólo que se evalúa un escalón más arriba. Y el corolario de §2.4 sigue valiendo tal cual: **si `/work/reports` queda vacío, no hay veredicto de éxito posible** — y eso sí lo puede verificar la tapa, porque es contar archivos, no interpretarlos.

Esto hay que decidirlo antes de tocar el endpoint. Es D16.

7. Qué se cae de nuestros documentos

7.1 Se corre la frontera de dominio

Todo el 03 está escrito asumiendo que el sandbox sabe Java. Bajo el sándwich no: compilar, testear y dictar el veredicto técnico son de la capa 2, y son opacos para nosotros. Pasamos de *corredor de Java con opinión* a *infraestructura agnóstica de lenguaje*.

Dejan de ser negociaciones nuestras las definiciones abiertas **1, 2, 4, 5, 6, 14, 16 y 17** del 03 §7. Entre ellas dos que costaron trabajo:

- **D12 / PAQUETE_RESERVADO** — no podemos validar un paquete si no sabemos qué es un paquete. (Atenuante: el README ya lo bajó de bloqueante, porque el orden del classpath lo contiene.)
- **D10 / captura de `stdout` por test** — era la única forma de no filtrar la entrada de un caso oculto. Pasa a depender de cómo G5 escriba su capa.

7.2 Se cae el contrato de 03 §6

Campo nuestro	Bajo el V4
<code>lenguaje: "JAVA_21"</code>	desaparece; lo define el perfil
<code>archivos[].rol</code> (FUENTE/TEST)	desaparece; mandan <code>files[] {path, content}</code> planos

Campo nuestro	Bajo el V4
<code>archivos[].visibilidad</code>	desaparece; los ocultos son "un archivo más"
<code>modo: VISIBLE\ COMPLETO</code>	no existe en el V4
<code>trazabilidad.suiteVersion</code> → <code>imagen</code>	lo reemplaza <code>profileId</code>
<code>limites</code> propuestos por T05	sobreviven, pero se mudan al perfil (§8)

Y se cae entera la respuesta de 03 §6.2: `resumen`, `tests[]`, `mensaje` y `visibilidad` por test. No parseamos el reporte: devolvemos lo que produjo la capa 2, normalizado por formato pero sin interpretación semántica.

7.3 Se aplana la máquina de estados

`TESTS_FALLIDOS`, `COMPILACION_FALLIDA` y `SUITE_INVALIDA` (03 §5.1) ya no las puede distinguir la tapa. Nos quedan los finales de infraestructura — `TIMEOUT`, OOM, salida anticipada, `ERROR_INTERNO`, `VEREDICTO_NO_CONFIA-BLE` — más "la capa 2 corrió y dijo X".

La distinción fina vuelve por las dos vías de §2.4: la banda 40–59 y `$SANDBOX_STATUS/fase`. Por eso las definiciones 10 y 12 (`ERROR_INTERNO` y `SUITE_INVALIDA` no consumen vida) siguen vivas: las calcula T05, pero nosotros le tenemos que dar con qué.

7.4 Queda obsoleto `otros/Contrato_Sandbox_Tema05.md`

Era Python 3.11, `source_code` único y `test_cases[]` con `expected_stdout`. El V4 es Java 21 con `files[]` y sin comparación de stdout. Sobrevive sólo la parte asíncrona y la tabla de estados (`infra_error` vs. falla del alumno).

8. El catálogo de perfiles

El §5 del V4 propone:

```
PUT /api/v1/sandbox/profiles/java21-junit
{ "name": "...", "image": "sandbox-java:21-tools", "entrypointScript": "#!/bin/sh\njavac ..." }
```

Es la evolución de nuestro §2.9 —ellos guardan el script en nuestra base en vez de elegir de una lista fija— y es la opción que recomendamos. Pero **tal como está escrita tiene cinco problemas**.

#	Problema	Por qué importa
1	<code>PUT</code> sobre el id: el perfil es mutable en el lugar	Dos ejecuciones con el mismo <code>profileId</code> se comportan distinto y no hay forma de saberlo. Mata la reproducibilidad de 03 §2.1 y es la misma trampa que D13
2	<code>image</code> es un string libre del cliente	Si el perfil deja elegir cualquier imagen, es la invariante P1 rota con otro nombre
3	Los límites siguen en el request	El perfil es justamente la cosa que sabe si esto es un JUnit o un PMD, que necesitan presupuestos distintos
4	No dice quién puede llamarlo	Es un endpoint de escritura que cambia qué código corre en nuestros contenedores

#	Problema	Por qué importa
5	El nombre <code>entrypointS-crypt</code> está mal	No es el entrypoint: el entrypoint es la capa 1 y es nuestro. Ese nombre reinstala la confusión que el propio §3 acaba de resolver

Cómo lo manejamos

Perfiles inmutables y versionados. `POST /profiles` crea una **versión nueva** y devuelve `{profileId, version, hash}`. `PUT` sobre una versión existente se rechaza. La ejecución referencia una versión exacta y guardamos el hash del script junto a cada ejecución. La reproducibilidad vuelve gratis, igual que con la `Idempotency-Key`.

Ciclo de vida: `BORRADOR` → `VALIDADA` → `ACTIVA` → `DEPRECADA`. Sólo una versión `ACTIVA` se ejecuta. Deprecar no rompe nada en vuelo, porque cada ejecución apunta a su versión.

El portón es un smoke test, no una lectura del script. Al registrar, corremos el perfil contra un bundle de referencia enlatado —una solución buena y una mala— y verificamos tres cosas: que deja algo en `/work/reports`, que devuelve un código de su banda, y que **los dos bundles no dan el mismo veredicto**. Si pasa, queda `VALIDADA`. Vale muchísimo más que cualquier revisión manual.

Los límites viven en el perfil, con dos presupuestos separados —compilación y pruebas— que es la salida (b) de §2.10.

El perfil trae su tabla de códigos de la banda `40-59`, y **declara el formato de reporte** (`junit-xml`, `pmd-xml`, ...), que es lo que le permite al worker aplicar D6 (§6):

```
{
  "name": "Java 21 - JUnit Runner",
  "imagenId": "sandbox-java-21-tools",
  "script": "#!/bin/sh\n...",
  "reportFormat": "junit-xml",
  "exitCodes": { "41": "No compila la solución", "42": "No compilan las pruebas" },
  "limits": { "compileMs": 20000, "evalCpuS": 10, "memoriaMb": 512 }
}
```

Cómo llega al contenedor: el worker busca el script en nuestra base y **lo escribe adentro del tar como** `/work/in/run.sh` antes de mandarlo por stdin. La capa 1 siempre hace lo mismo, `cd /work/in && sh ./run.sh`, sin enterarse de dónde salió.

La respuesta a "ustedes van a poder auditar los scripts"

El V4 ofrece la auditoría como ventaja para nosotros. **No conviene aceptar ese encuadre**, porque nos compromete a algo que no podemos sostener: analizar shell estáticamente no sirve de mucho, y una auditoría que nadie hace no es un control, es un papel.

La respuesta correcta es más fuerte:

No necesitamos auditar el script para que sea seguro. **El script es capa 2, y la capa 2 ya está tratada como no confiable:** corre sin red, sin root, con `/libs` de sólo lectura, con límite de CPU y memoria, encerrada entre dos tapas que no controla y sin acceso al nonce. Un `run.sh` malicioso no puede hacer nada que el código del alumno no pudiera hacer ya.

Por eso podemos aceptar la Opción 1 sin cargarnos una responsabilidad nueva. La revisión que sí hacemos es **ope-**

rativa, y la hace el smoke test solo.

9. Las opciones 1 y 3 son la misma opción

Bajo la Opción 1 el script vive en nuestra base y el worker lo "inyecta virtualmente". La única forma sensata de inyectarlo es meterlo en el tar. Es decir: **a nivel del contenedor, las Opciones 1 y 3 son idénticas** —un `run.sh` en `/work/in` — y sólo se diferencian en quién guarda el script. La Opción 2 se expresa como un `run.sh` generado a partir del array.

Dos consecuencias:

1. **Un solo mecanismo en el entrypoint sirve para las tres.** No hay que elegir para poder empezar a construir.
2. La discusión "cuál de las tres" pasa a ser de **almacenamiento y auditoría**, que es donde la Opción 1 gana sola.

Conviene decírselo: les baja el costo percibido de aceptar la 1, porque deja de ser una puerta que se cierra.

10. Cómo se prueba que no rompimos nada

El criterio de aceptación se escribe solo. El pipeline de Java que hoy está cableado en el entrypoint **se reescribe como el primer `run.sh` de referencia** —lo escribimos nosotros y se lo entregamos al Grupo 5 como plantilla— y los casos que ya tenemos tienen que dar exactamente el mismo resultado:

Caso de prueba	Qué prueba	Hoy	Con el sándwich
<code>ok-suma</code>	una entrega correcta	aprueba, 3 pruebas, 0 fallas	idéntico
<code>hostil-exit0</code>	el alumno apaga la máquina virtual antes de que corran las pruebas	salida anticipada (29)	idéntico
<code>hostil-cpu</code>	el alumno se cuelga en un bucle infinito	timeout	idéntico
<code>hostil-reporte-loop</code>	el alumno reescribe el reporte en bucle	veredicto no confiable (30)	idéntico
<code>symlink</code>	el tar trae un enlace que apunta fuera de la carpeta	<i>no existe todavía</i>	paquete inválido (22)
<code>hardlink</code>	ídem, con enlace duro	<i>no existe todavía</i>	paquete inválido (22)

Si los cuatro primeros dan igual, la refactorización es correcta por construcción. Recién ahí se le pasa el timón al Grupo 5.

Los dos últimos son deuda que hay que pagar antes, no después. Hoy nuestro validador solo acepta rutas que empiecen con `src/` o `test/`; con `run.sh` en la raíz y configuraciones del Grupo 5 en cualquier lado, esa lista blanca no sobrevive, y hay que reemplazarla por prohibiciones sobre el **tipo** de entrada del tar. Mientras la lista era angosta el hueco de los enlaces era estrecho; abriendo el árbol, deja de serlo.

11. Trabajo que aparece y trabajo que se borra

Aparece

Ítem	Tamaño
CRUD de perfiles con versionado inmutable + estados + autenticación servicio-a-servicio	Mediano. Familia de endpoints nueva en el 07
Pipeline del smoke test de perfiles con bundles de referencia	Mediano. Reusa <code>scripts/integracion-runner.mjs</code>
Catálogo de imágenes base construidas por nosotros (<code>/libs</code> , no-root) + su nombrado y CI	Mediano
Generalizar <code>entrypoint.sh</code> : sacar pasos 2–5, meter la invocación de <code>run.sh</code> , generalizar el sobre	Chico. La estructura ya es la del sándwich
Mudar la regla de D6 al worker, indexada por formato de reporte	Chico-mediano
Casos hostiles de symlink y hardlink (P2) y validación de <code>ruta</code> en la API	Ya estaban pendientes

Se borra

- Todo el conocimiento de Java del entrypoint: `javac` en dos fases, derivación de nombres de clase, `--select-class` , `--include-classname` . **Se va al `run.sh` de G5.**
- La negociación de `suiteVersion` → `imagen` .
- Las definiciones abiertas 1, 2, 4, 5, 6, 14, 16 y 17 del 03 §7.

Sobrevive intacto

Los dos ejecutores (Java 58 tests, Node 65 tests) son **transporte, no lenguaje**: no los toca nada de esto salvo de dónde salen la imagen y los límites. La suite hostil, los invariantes de 08 §12 y el aislamiento tampoco se mueven — y ganan peso relativo, porque pasan a ser casi lo único que aportamos al veredicto.

12. Decisiones nuevas para el README

Para agregar a la tabla de abiertas cuando hagamos la actualización general:

#	Decisión	Quién la cierra
D16	¿Dónde se verifica "hay evidencia real" cuando la tapa no sabe leer el reporte? Propuesta: se muda al worker, con el formato declarado por el perfil (§6). Reemplaza a D6 bajo el sándwich y hay que cerrarla antes de tocar el entrypoint	Nosotros
D17	¿Aceptamos la Opción 1? Propuesta: sí, con perfiles inmutables y versionados	Nosotros + T05
D18	¿Los límites viven en el perfil o en el request? Propuesta: en el perfil, con presupuestos separados (§2.10, salida (c) como piso y (b) como opción)	Nosotros + T05

#	Decisión	Quién la cierra
D19	Bandas de códigos de salida (20-31 nuestros, 40-59 de ellos, hueco 32-39)	Nosotros + T05
D20	Modelo asincrónico: el V4 no lo menciona. El resultado viaja como evento <code>EjecucionFinalizada</code> por el bus de la plataforma, que es Kafka y lo implementa el grupo de notificaciones . Falta fijar el nombre del evento/tópico con T05 y si quieren el <code>GET</code> como respaldo. Pendiente de contexto: 07 §3.4 tiene el relay publicando a RabbitMQ , que es nuestra cola de trabajo interna (04 §3). Cómo se articula eso con el bus de la plataforma depende de decisiones de integración que exceden a este documento; se resuelve cuando tengamos el panorama completo de eventos, no acá	Nosotros + T05
D21	Catálogo de imágenes: quién las nombra, quién las versiona, quién aprueba una nueva. Incluye si la base pasa a ser Alpine —nuestro endpoint está en bash y el shell reducido de Alpine no lo corre tal cual—	Nosotros

Y las cinco preguntas que quedan del contrato, que **ninguna se resuelve rehaciendo código nuestro**:

#	Pregunta abierta	Quién la contesta
A1	¿El límite de CPU es por proceso o agregado? (§2.10)	Nosotros, con un caso de prueba
A2	¿Cuánta CPU y memoria consumen realmente PMD y ArchUnit?	El Grupo 5 , con mediciones
A3	¿Qué etiquetas de fase y qué códigos 40-59 definen?	El Grupo 5
A4	¿Cuántos perfiles arrancamos, y con qué contenido?	Los dos
A5	¿El botón "Ejecutar" del IDE pasa por el sandbox? (definición 8 del 03 §7)	T05. Sigue abierta y cambia el dimensionamiento por completo; el V4 no la toca

Un pendiente técnico nuevo:

#	Pendiente
P9	Refactor del <code>entrypoint.sh</code> al sándwich: sacar los pasos 2-5, invocar <code>run.sh</code> , generalizar el sobre (<code>exitCodeJava</code> , <code>clasesTest</code> y <code>testsEnReporte</code> dejan de tener sentido como campos fijos). Bloqueado por D16

13. Qué hacemos ahora

1. **Mandar la respuesta** (`otros/Respuesta_G8_a_Propuesta_V4.md`). Es lo más urgente: contiene el contrato de §2, que es lo que ellos necesitan para escribir su capa.
2. **Cerrar D16 entre nosotros**, porque no depende de ellos y bloquea el refactor.
3. **Pagar la deuda de P2** (symlink y hardlink), que con el árbol de archivos abierto deja de ser un hueco estrecho.
4. **Arrancar P9:** el refactor del endpoint es el mismo en las tres opciones (§9), así que no hace falta esperar la confirmación de la Opción 1. Sí hay que esperar D16.
5. **El CRUD de perfiles** después de la confirmación.
6. **Actualización exhaustiva de los documentos** — 03 , 07 , 08 , `README` — cuando la respuesta tenga contestación. Antes no: quedarían escritos contra un contrato que todavía se mueve.

Documentos relacionados

- `otros/Respuesta_G8_a_Propuesta_V4.md` — lo que se les manda.
- `10-propuesta-g5-sandwich.md` — versión extendida y divulgativa de la §2, con el glosario y los cuatro tipos de desafío desarrollados.
- `03-ms-sandbox-ejecucion.md` — el contrato que este cambio rompe.
- `08-spec-ejecutor.md` §7 — nonce, entrada y separación del reporte.
- `README.md` — estado de decisiones.